ENSEA
Beyond Engineering

# Développement d'un robot autonome pour la détection et l'élimination des mauvaises herbes par vision artificielle

**Projet 2A**
**Professeur encadrant : Ayoub Jebri**

FERHANI ARES - KERD MEHDI - SBAI NADA - REGRAG SALMA - LE CORRE SOËL - BECHAINE DOUAE

**ENSEA**
**2025-2026**

# Table des matières

**1 Introduction** **3**
1.1 Description du projet . . . 3
1.2 Organisation du travail en équipe . . . 3
1.3 Cahier des Charges Fonctionnelles (CDCF) . . . 4
1.4 Avancée du projet au S7 . . . 5
1.5 Avancée du projet au S8 . . . 6

**2 Détection de l'image** **7**
2.1 Acquisition des images : choix de l'ESP32-CAM . . . 7
2.2 Pré-traitement : calibration de la caméra et correction de la distorsion . . . 7
2.3 Choix de l'approche et des outils . . . 9
2.4 Jeu de données et préparation . . . 10
2.4.1 Augmentation et enrichissement du jeu de données . . . 11
2.5 Entraînement du modèle YOLOv8 . . . 11
2.6 Détection sur le flux vidéo . . . 13
2.7 Bilan de la chaîne de vision . . . 13

**3 Définition de la position cible** **14**
3.1 Architecture matérielle : Vision embarquée . . . 14
3.2 Hypothèse de travail et modélisation . . . 14
3.3 Chaîne de traitement de l'information (Data Pipeline) . . . 15
3.4 Intégration logicielle et connectivité . . . 15
3.5 Stratégie de calibration et homographie . . . 15
3.6 Stratégie de commande et gestion de l'altitude . . . 16

**4 Fabrication d'un support caméra par impression 3D** **16**
4.1 Conception du support . . . 16
4.2 Fabrication par impression 3D . . . 16
4.3 Intégration sur le robot . . . 17

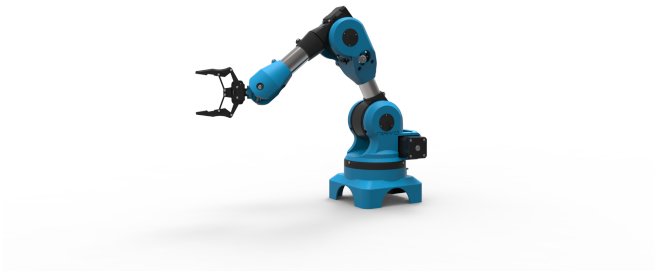
**5 Bilan final du projet** **17**
5.1 Travaux réalisés sur la phase finale . . . 17
5.2 Architecture finale obtenue . . . 18
5.3 Résultats obtenus . . . 18
5.4 Limites observées . . . 18
5.5 Conclusion générale . . . 18

# 1 Introduction

## 1.1 Description du projet

Le projet consiste à développer un robot capable de détecter les mauvaises herbes dans une plantation à l'aide d'un système d'analyse d'images provenant de caméras. Équipé d'un bras manipulateur, le robot doit être en mesure d'arracher ces mauvaises herbes sans endommager les cultures. Si l'avancement du projet le permet, un second mode pourra être intégré : celui-ci permettra au robot de récolter les fruits ou légumes lorsqu'ils seront détectés comme étant mûrs.
Le bras utilisé est celui du robot **Niryo One**.

Le cœur de notre projet repose sur la vision par ordinateur, en particulier sur la détection de mauvaises herbes via **YOLO** et **OpenCV**. L'écosystème standard de l'intelligence artificielle et de la vision étant fortement orienté vers Python, ce langage s'est imposé naturellement pour l'implémentation de l'ensemble de la chaîne de traitement. Ce choix a permis de simplifier l'intégration entre les briques de détection, de traitement géométrique et de commande du robot.





## 1.2 Organisation du travail en équipe

L'une des premières tâches réalisées lors de la prise en main de ce projet a été de créer un environnement collaboratif de travail car, comme nous avons pu le remarquer lors de nos projets de première année, la **communication** est la clé d'un projet organisé et réussi.
Nous avons donc opté pour la création d'un **Google Drive** regroupant les ressources nécessaires à la compréhension et à la prise en main du robot (datasheets, TP de troisième année, manuels d'utilisation, etc.).
À cela s'ajoute la création d'un **groupe WhatsApp** pour des échanges plus directs et rapides, notamment concernant la préparation des séances de projet ou tout simplement pour notifier l'ajout d'un document sur le Drive.

Lors des premières séances, où la vision globale de notre projet n'était pas encore totalement nette, nous avons réparti notre travail selon trois grands axes :
- **Détection de l'image :** Salma et Mehdi
- **Définition de la position cible pour le bras :** Douae et Ares
- **Calibration caméra/robot :** Nada et Soel

Si ces axes correspondent bien aux grandes briques du projet, ils ne demeurent pas indépendants les uns des autres. Un axe a besoin des résultats de l'autre pour pouvoir être abordé correctement. Nous avons ainsi rapidement fait évoluer l'organisation afin d'optimiser notre efficacité : les axes ont été traités dans l'ordre logique du projet, tout en étant découpés en sous-axes répartis entre les différents duos.



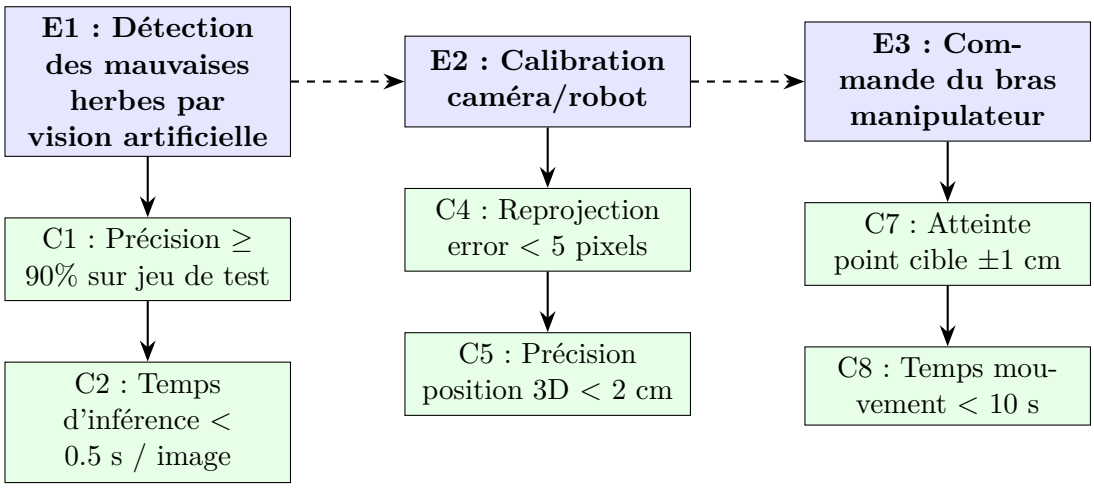



## 1.3 Cahier des Charges Fonctionnelles (CDCF)

L'énoncé du projet ne définissant pas d'objectifs précis si ce n'est le but final, nous avons dû nous questionner en premier lieu sur les critères qui feraient de notre travail un projet abouti.
Nous avons donc défini des objectifs réalisables avec le matériel mis à notre disposition, tant sur le plan informatique et IA concernant la détection des mauvaises herbes que sur l'aspect déplacement du bras.
Ces objectifs sont regroupés sous la forme de critères et d'exigences à respecter, présentés dans le **cahier des charges fonctionnelles** ci-dessous :

**Cahier des Charges Fonctionnel (CDCF)**

*Robot autonome de désherbage par vision artificielle*




**Exigence E1**
Détection robuste via caméra. Utilisation d'un modèle YOLO/CNN entraîné sur données spécifiques.

**Exigence E2**
Transformation des coordonnées image vers le repère robot via matrice de passage.

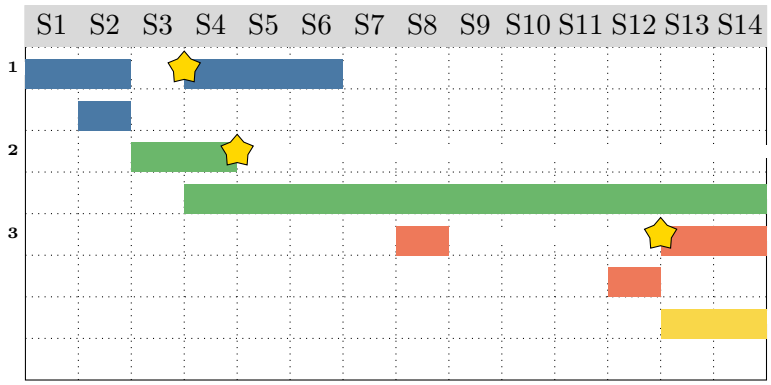
**Exigence E3**
Calcul des angles articulaires et gestion des trajectoires pour atteindre la cible.

Ainsi, lors de chaque début et fin de séance nous faisions le point sur l'avancée du projet et notamment sur les axes de travail précédemment définis, en vérifiant le respect ou non des différents critères du cahier des charges.

## 1.4 Avancée du projet au S7

Le diagramme de Gantt ci-dessous présente l'avancée de notre travail durant le premier semestre.

# Diagramme de Gantt : Robot détecteur et ramasseur de mauvaises herbes (sept-déc 2025)




## Etapes importantes ()

1. **Élaboration du CDCF (S3)** : Finalisation du Cahier des Charges Fonctionnel.
2. **Décision Technologique (S4)** : Choix définitif du langage Python et de l'algorithme YOLO.
3. **Réception Caméra (S12)** : Livraison du matériel nécessaire pour la phase finale.

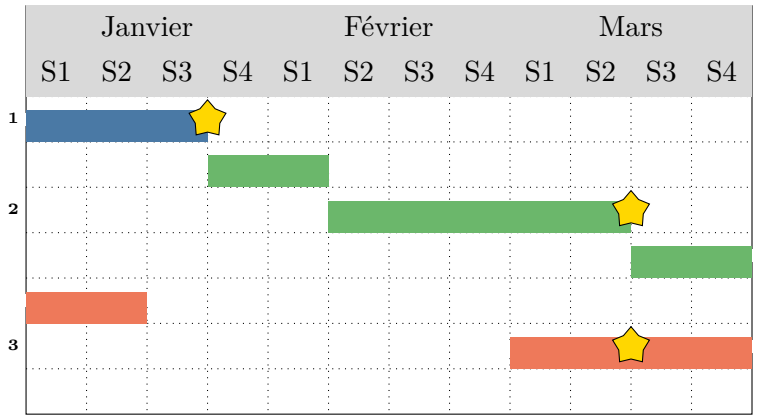
## Légende

**ANALYSE & RECHERCHE** : Découverte, recherches, répartition du travail, travail de documentation.
**DÉVELOPPEMENT LOGICIEL / IA** : NiryoStudio, entraînement YOLO et programmation Python.
**MATÉRIEL & CAO** : Commande matériel, mesures et conception du support 3D.
**DOCUMENTATION** : Rédaction et mise en forme du rapport final.

## 1.5 Avancée du projet au S8

Le diagramme de Gantt ci-dessous présente l'avancée de notre travail durant le second semestre.

# Diagramme de Gantt : Robot détecteur et ramasseur de mauvaises herbes (Janvier - Mars 2026)




## Etapes importantes ()

1. **Validation Homographie (Jan S3)** : Calcul des coordonnées bras/caméra validé.
2. **Calibration Optique (Mar S2)** : Correction de la distorsion pour la précision visuelle.
3. **Optimisation Matérielle (Mar S2)** : Adaptation de la pince et autonomie batterie.

## Légende

**ANALYSE & ALGORITHMIE** : Étude de l'homographie et positionnement spatial.
**INTELLIGENCE ARTIFICIELLE** : YOLO (données, entraînement) et pré-traitement (calibration).
**MATÉRIEL & TESTS** : Intégration caméra, tests robotiques et modifications physiques.

# 2 Détection de l'image

La détection visuelle constitue une brique centrale du projet, puisqu'elle permet d'identifier automatiquement la présence et la position des mauvaises herbes à partir d'images acquises dans un environnement agricole. Cette information est ensuite exploitée pour déclencher et guider l'action du bras manipulateur. Cette section présente la chaîne de détection retenue, depuis l'acquisition des images jusqu'à l'application du modèle entraîné sur un flux vidéo en temps réel.

## 2.1 Acquisition des images : choix de l'ESP32-CAM

L'acquisition des images est assurée par un module **ESP32-CAM**. Ce composant compact intègre à la fois un microcontrôleur et un capteur caméra, ainsi qu'une interface Wi-Fi.

Le choix de l'ESP32-CAM se justifie par plusieurs avantages :

— transmission sans fil du flux vidéo via Wi-Fi,

— faible encombrement, compatible avec une intégration sur un bras robotique,

— coût réduit et large diffusion dans les projets *IoT*,

— simplicité de mise en œuvre pour des projets académiques et prototypes.

Ces caractéristiques en font une solution adaptée aux projets de robotique expérimentale, où la flexibilité et la rapidité de déploiement sont prioritaires.




FIGURE 1 – Module ESP32-CAM utilisé pour l'acquisition des images

## 2.2 Pré-traitement : calibration de la caméra et correction de la distorsion

Avant d'exploiter les images pour la détection des mauvaises herbes, une étape de **pré-traitement** a été mise en place afin d'améliorer la qualité géométrique des images acquises. En effet, les caméras embarquées de petite taille, comme celle intégrée à l'ESP32-CAM, introduisent généralement des déformations optiques non négligeables, en particulier en périphérie de l'image. Ces distorsions peuvent

fausser la position apparente des objets observés et dégrader la précision des traitements de vision qui suivent.

Dans notre projet, un travail de **calibration de la caméra** a donc été réalisé à l'aide de la bibliothèque **OpenCV**. Cette procédure a pour objectif d'estimer les paramètres intrinsèques de la caméra, en particulier la **matrice de calibration**, ainsi que les **coefficients de distorsion**. La connaissance de ces paramètres permet ensuite d'appliquer une opération de **correction géométrique** (*undistortion*) afin d'obtenir une image plus fidèle à la scène réelle.

La calibration repose sur l'utilisation d'un motif de référence de type **damier**, dont la géométrie est connue. Plusieurs images du damier sont acquises sous différents angles et positions, puis OpenCV détecte automatiquement les coins internes du motif. À partir de ces correspondances entre les points théoriques du damier dans l'espace réel et leur projection dans l'image, l'algorithme estime les paramètres de la caméra.

Sur le plan mathématique, le passage d'un point 3D de coordonnées $(X, Y, Z)$ dans le repère caméra vers son image de coordonnées pixel $(u, v)$ s'exprime à partir de la matrice intrinsèque :

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

où :

— $f_x$ et $f_y$ représentent les focales exprimées en pixels selon les axes horizontal et vertical,

— $c_x$ et $c_y$ représentent les coordonnées du centre optique dans l'image,

— le terme de cisaillement est supposé nul ici, ce qui est le cas classique pour ce type d'application.

En coordonnées homogènes, la projection idéale sans distorsion peut s'écrire sous la forme :

$$s \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \begin{pmatrix} R & t \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

où $R$ est la matrice de rotation, $t$ le vecteur de translation et $s$ un facteur d'échelle. Dans le cadre de la calibration, OpenCV estime donc à la fois les paramètres **intrinsèques** de la caméra et, pour chaque image du damier, les paramètres **extrinsèques** décrivant la pose du damier par rapport à la caméra.

En pratique, cette projection idéale est perturbée par les défauts optiques de la lentille. Les plus importants sont les phénomènes de **distorsion radiale** et de **distorsion tangentielle**. Ils sont modélisés par un ensemble de coefficients, généralement notés :

$$(k_1, k_2, k_3, p_1, p_2)$$

avec :

— $k_1$, $k_2$, $k_3$ : coefficients de **distorsion radiale**, responsables notamment de l'effet de bombement ou de pincement visible sur les bords de l'image,

— $p_1$, $p_2$ : coefficients de **distorsion tangentielle**, liés à un défaut d'alignement entre la lentille et le capteur.

À partir des coordonnées normalisées idéales $(x, y)$, la forme corrigée par distorsion s'exprime de manière classique par :

$$r^2 = x^2 + y^2$$

$$x_d = x\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) + 2p_1 xy + p_2(r^2 + 2x^2)$$

$$y_d = y\left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6\right) + p_1(r^2 + 2y^2) + 2p_2 xy$$

où $(x_d, y_d)$ correspondent aux coordonnées affectées par les défauts optiques. L'opération d'*undistortion* consiste alors à utiliser les paramètres estimés pour reconstruire une image géométriquement plus fidèle.

Cette étape est particulièrement importante dans notre cas, car la détection visuelle ne constitue pas une fin en soi : les positions extraites dans le repère image doivent ensuite être exploitées pour guider le bras robotique. Il est donc nécessaire de limiter autant que possible les erreurs liées à l'optique afin d'améliorer la cohérence entre la perception visuelle et la position réelle de la cible.

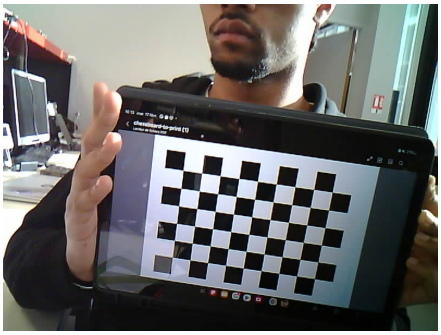
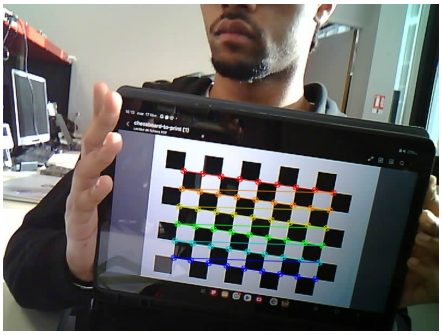
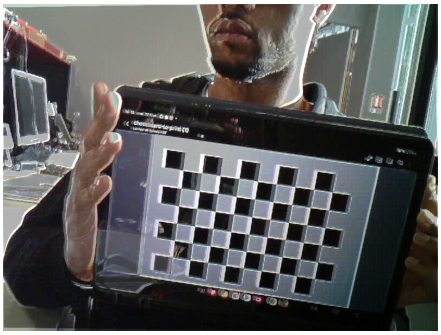


Image du damier utilisée pour la calibration



Détection automatique des coins du damier



Superposition avant/après correction pour visualiser les écarts

FIGURE 2 – Étapes principales de la calibration de la caméra et mise en évidence de la correction de distorsion

À l'issue de cette procédure, nous obtenons donc une **matrice intrinsèque** propre à la caméra, ainsi que les **coefficients de distorsion** associés aux images acquises. Ces résultats numériques sont directement fournis par OpenCV à partir des images de calibration, puis réutilisés dans le pipeline de traitement afin de redresser automatiquement les images avant leur exploitation.

```
=== CALIBRATION CHARGÉE ===
Camera matrix :
 [[756.328     0.      387.09558]
 [  0.      755.1176  308.0407 ]
 [  0.        0.        1.     ]]

Distortion coefficients :
 [[-0.06924839  0.3351038  -0.0038068  -0.00957238 -0.39399996]]
```

FIGURE 3 – Exemple de matrice intrinsèque et de coefficients de distorsion obtenus à partir de nos images de calibration

La correction appliquée permet de réduire les déformations visibles, notamment sur les lignes droites situées en bord de champ, et d'obtenir une représentation plus rigoureuse de la scène observée. Ce pré-traitement constitue donc une étape essentielle de la chaîne de vision. Il permet de fiabiliser les mesures extraites à partir des images et prépare plus efficacement les étapes suivantes, qu'il s'agisse de la détection par intelligence artificielle ou de l'estimation de position nécessaire au pilotage du robot.

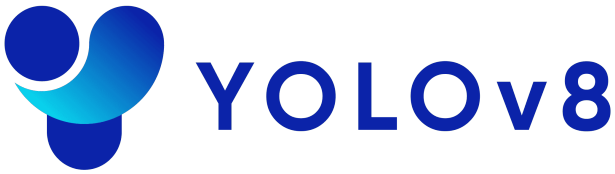
## 2.3 Choix de l'approche et des outils

Afin de répondre aux contraintes de robustesse et de traitement en temps quasi réel imposées par une application robotique, nous avons opté pour une approche par **détection d'objets** basée

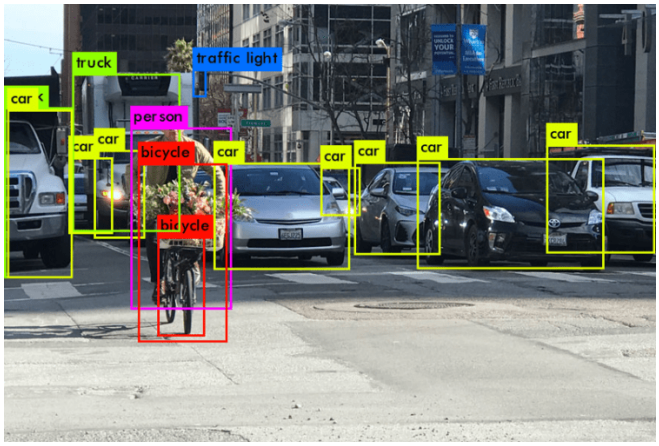
sur l'algorithme **YOLOv8**.

Les modèles de la famille YOLO sont largement utilisés dans le domaine de la robotique mobile et de la robotisation industrielle. Leur popularité s'explique par leur rapidité d'inférence, leur architecture compacte et la grande quantité de ressources techniques disponibles. YOLOv8, développé par Ultralytics, constitue une version récente et optimisée de cet algorithme, offrant un bon compromis entre précision et vitesse d'exécution, ce qui le rend particulièrement adapté aux systèmes embarqués et aux projets robotiques académiques.

L'ensemble du développement est réalisé en **Python**, langage de référence pour les applications de vision par ordinateur et d'intelligence artificielle, permettant une intégration simple avec les bibliothèques de traitement d'images et les outils de déploiement du modèle.




Logo YOLOv8 (Ultralytics)




Exemple de détection d'objets avec boîtes englobantes

FIGURE 4 – Illustration du principe de la détection d'objets avec YOLO

## 2.4 Jeu de données et préparation

Le modèle de détection est entraîné à partir d'un jeu de données issu de la plateforme **Roboflow**, dédiée à la gestion, l'annotation et la préparation de jeux de données pour la vision par ordinateur.

Un jeu de données complet destiné à l'apprentissage supervisé est généralement structuré en trois sous-ensembles distincts :

- **Ensemble d'entraînement (train)** : utilisé pour ajuster les paramètres du modèle.
- **Ensemble de validation (validation)** : utilisé pour évaluer les performances du modèle durant l'entraînement et éviter le surapprentissage.
- **Ensemble de test (test)** : utilisé pour évaluer le comportement final du modèle sur des données jamais vues.

Chaque image du dataset est associée à des annotations encodant la classe de l'objet détecté et les coordonnées de la boîte englobante correspondante dans le repère image.

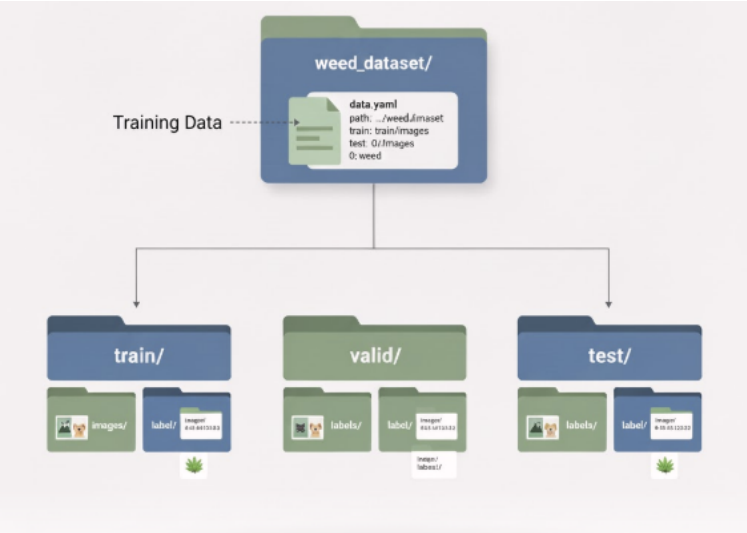



FIGURE 5 – Organisation typique d'un jeu de données pour l'apprentissage supervisé : ensembles d'entraînement, de validation et de test

### 2.4.1 Augmentation et enrichissement du jeu de données

Afin d'améliorer la robustesse du modèle face aux variations rencontrées en conditions réelles, des techniques d'augmentation de données sont appliquées lors de la préparation du dataset. Ces transformations permettent de simuler artificiellement des situations variées sans nécessiter l'acquisition de nouvelles images :

— ajout de bruit,
— variations de luminosité et de contraste,
— rotations et changements d'échelle,
— inversions horizontales.

Au-delà de cette augmentation artificielle, une amélioration importante du projet final a consisté à **enrichir le jeu de données avec de nouvelles images acquises expérimentalement**, puis à **les annoter manuellement** avant de relancer l'entraînement du modèle. Cette étape a permis d'introduire des scènes plus proches des conditions réelles d'utilisation du système, et donc d'améliorer la stabilité globale de la détection.

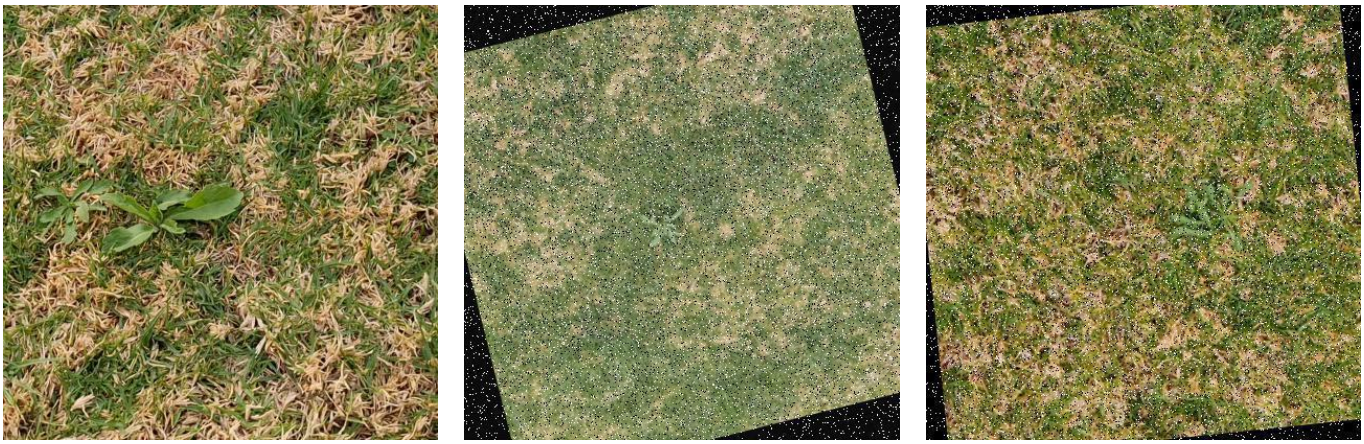


FIGURE 6 – Exemples d'images issues du jeu de données

## 2.5 Entraînement du modèle YOLOv8

L'entraînement du modèle repose sur le principe de la détection monocoup (*one-stage detection*). Contrairement aux approches en deux étapes, YOLO prédit simultanément les classes et les positions

des objets sur l'ensemble de l'image.

L'entraînement est réalisé à partir d'un modèle pré-entraîné, permettant de réutiliser des caractéristiques visuelles génériques avant un ajustement spécifique à la détection des mauvaises herbes. Cette approche réduit le temps d'apprentissage et améliore les performances du modèle dans un contexte de dataset limité.

Dans la phase finale du projet, cette étape n'a pas été figée après un premier entraînement : le modèle a été **réentraîné** après enrichissement du dataset afin d'améliorer la qualité des détections et de mieux prendre en compte les cas rencontrés lors des essais réels.

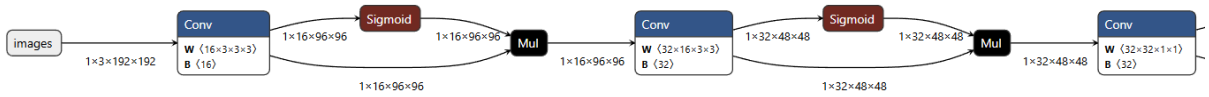



FIGURE 7 – Extrait de l'architecture globale du modèle entraîné et exemples de détection des mauvaises herbes

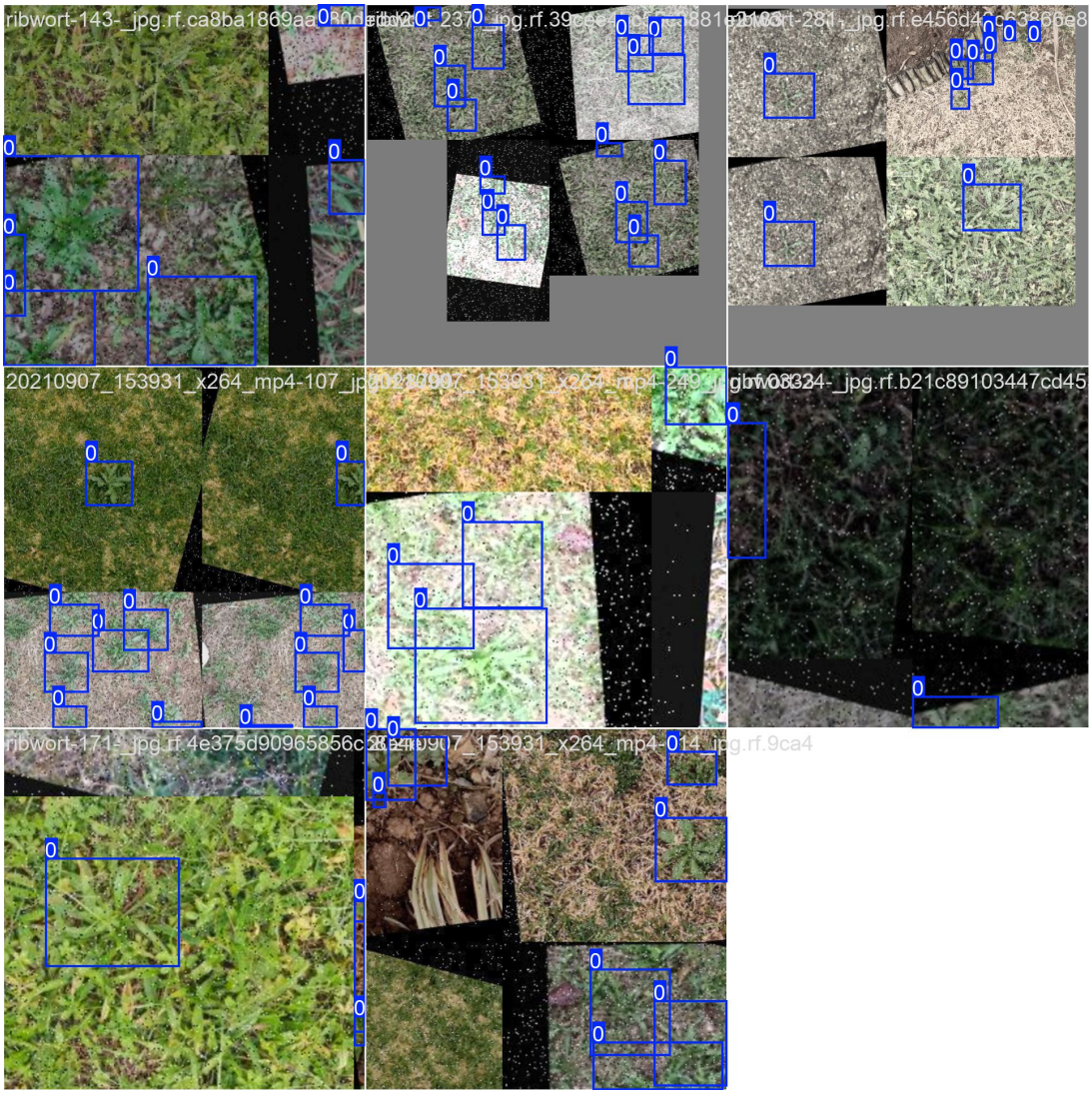


FIGURE 8 – Exemples de mauvaises herbes détectées par le modèle

## 2.6 Détection sur le flux vidéo

Le flux vidéo transmis par l'ESP32-CAM est récupéré sur un PC central, chargé d'exécuter le traitement de vision par ordinateur. Les images sont extraites du flux et analysées en temps réel par le modèle YOLOv8 entraîné. Pour chaque image, le système détecte les mauvaises herbes présentes et fournit leur localisation sous forme de boîtes englobantes dans le repère image.

Ces informations constituent l'entrée des modules de calcul de position et de commande du bras robotique, permettant la transition entre perception visuelle et action physique.

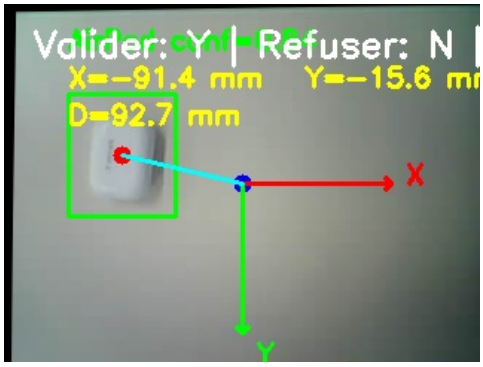



FIGURE 9 – Résultat de la détection après capture via ESP32-CAM. L'image illustre également la transformation par homographie, détaillée dans la suite du rapport

## 2.7 Bilan de la chaîne de vision

La chaîne complète de vision a été menée jusqu'à un niveau fonctionnel cohérent avec les objectifs du projet. Par rapport à l'état initial du travail, les apports les plus significatifs ont été l'ajout d'un pré-traitement optique par calibration, l'enrichissement du jeu de données avec de nouvelles images annotées, puis le réentraînement du modèle de détection. Ces améliorations ont permis de rendre la détection plus robuste et plus exploitable dans une logique de commande robotique.

# 3 Définition de la position cible

La réussite de la tâche de désherbage repose sur la capacité du système à traduire une information visuelle (pixels) en une commande motrice précise (coordonnées métriques). Cette section détaille l'architecture matérielle et la stratégie algorithmique mises en œuvre pour opérer ce changement de repère spatial.

## 3.1 Architecture matérielle : Vision embarquée

Contrairement aux solutions classiques utilisant une webcam filaire, nous avons opté pour une architecture de vision déportée basée sur un module **ESP32-CAM**. Ce choix technique répond à une contrainte mécanique forte :

— **Configuration Eye-in-Hand :** la caméra est fixée directement sur l'effecteur du robot pour être au plus près de l'action.

— **Liberté cinématique :** la transmission du flux vidéo par Wi-Fi supprime la liaison filaire entre la caméra et le poste de traitement, évitant que le câblage ne gêne les rotations du bras manipulateur.

## 3.2 Hypothèse de travail et modélisation

Le problème fondamental de la vision par ordinateur est la perte de l'information de profondeur lors de la projection d'un monde 3D sur un capteur 2D. Cependant, dans le cadre spécifique de ce projet, nous pouvons poser une **hypothèse de planéité** :

— le référentiel de la base du robot et les cibles reposent sur le même plan physique ;

— la prise de vue s'effectue toujours depuis une pose d'observation fixe et connue.

Cette contrainte physique permet de simplifier le modèle : une transformation projective plane (**homographie**) suffit pour mapper les coordonnées pixels vers des coordonnées métriques sur le plan de travail.

**Validation de l'hypothèse :** bien que les végétaux possèdent un volume, l'erreur de projection géométrique reste limitée grâce à une prise de vue quasi-verticale et demeure acceptable au regard des tolérances mécaniques de la pince.

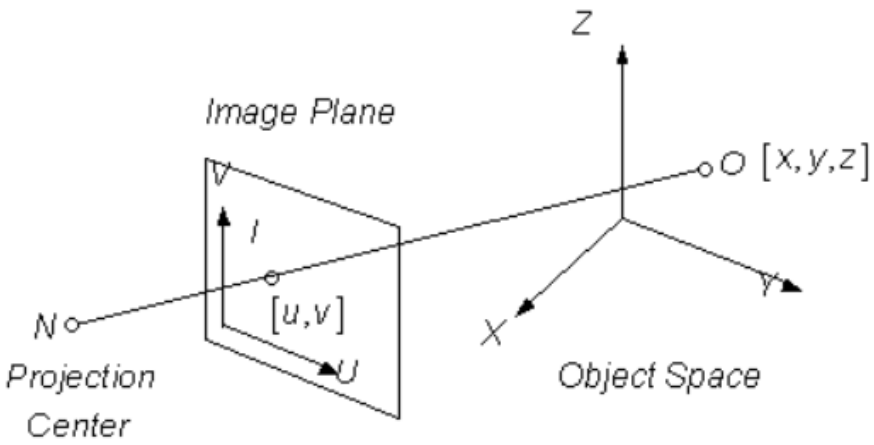



FIGURE 10 – Modèle de projection : passage du repère monde $(X, Y, Z)$ au repère image $(u, v)$. Dans notre cas, l'hypothèse de planéité revient à fixer $Z = 0$

## 3.3 Chaîne de traitement de l'information (Data Pipeline)

L'algorithme de commande suit le flux de données présenté dans la figure 11. Il illustre le passage de l'espace image à l'espace opérationnel du robot via un traitement déporté sur le PC central.

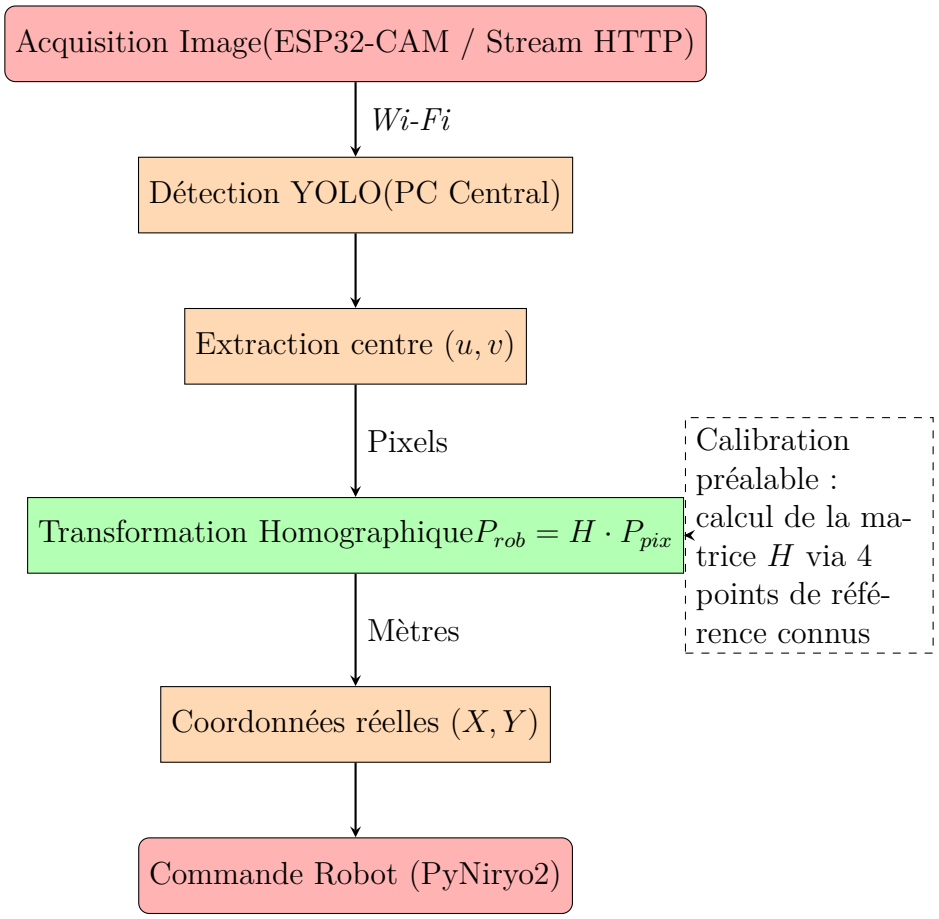



FIGURE 11 – Flux de données : de la détection visuelle à l'actionnement

## 3.4 Intégration logicielle et connectivité

L'architecture finale repose sur un **PC central** jouant le rôle de nœud de calcul et de coordination. Ce poste reçoit le flux vidéo transmis par la caméra via **Wi-Fi**, exécute les traitements de vision et de géométrie, puis envoie les consignes de mouvement au robot Niryo via une liaison **Ethernet**.

Cette séparation des flux permet d'adapter chaque lien à son usage :

— le **Wi-Fi** est utilisé pour la transmission souple du flux vidéo issu de la caméra embarquée ;

— l'**Ethernet** est utilisé pour assurer une communication plus stable avec le robot lors de la phase de commande.

Le PC central assure ainsi l'enchaînement complet des traitements : réception de l'image, détection par le modèle YOLOv8, calcul du point cible dans le repère robot, puis transmission de la commande au bras manipulateur. Cette architecture a permis de valider la faisabilité d'une chaîne unifiée allant de la perception à l'action.

## 3.5 Stratégie de calibration et homographie

Pour définir la matrice de passage $H$ reliant le repère image $\mathcal{R}_{img}$ (pixels) au repère robot $\mathcal{R}_{rob}$ (mètres), nous avons mis en place une procédure de calibration extrinsèque manuelle.

Nous utilisons l'algorithme DLT (*Direct Linear Transformation*) via la méthode des 4 points coplanaires. Ces points servent de références métriques connues pour résoudre le système d'équations

linéaires définissant la matrice $H$. Une fois la matrice calculée, tout point pixel $P_{pix} = [u, v, 1]^T$ détecté par le réseau de neurones est converti en point réel $P_{rob}$ par produit matriciel, permettant à l'API `PyNiryo2` de planifier la trajectoire d'extraction.

### 3.6 Stratégie de commande et gestion de l'altitude

L'algorithme de vision par homographie ne fournissant que des coordonnées planaires $(X, Y)$, l'information de hauteur des végétaux n'est pas reconstruite. Pour pallier cette limitation, nous avons mis en place une stratégie de commande en boucle ouverte sur l'axe vertical :

1. **Approche :** le robot se positionne à la verticale de la cible $(X, Y)$ à une hauteur de sécurité.
2. **Descente :** l'effecteur descend à une altitude de saisie fixe, déterminée expérimentalement.
3. **Extraction :** après fermeture de la pince, le robot effectue une remontée verticale.

Cette stratégie simple a permis de mettre en place une première commande cohérente du bras à partir d'une cible détectée visuellement, sans recourir à une reconstruction 3D complète de la scène.

## 4 Fabrication d'un support caméra par impression 3D

Afin d'intégrer la caméra ESP32-CAM au système robotique, un support mécanique spécifique a été conçu et fabriqué par impression 3D. Ce support permet de fixer solidement la caméra sur le robot Niryo tout en assurant une orientation adaptée à la zone d'observation.

### 4.1 Conception du support

Le support a été modélisé à l'aide d'un logiciel de conception assistée par ordinateur, puis exporté au format `STL`, couramment utilisé pour l'impression 3D.

Il prend la forme d'une **boîte rigide** dans laquelle la caméra ESP32-CAM est placée, assurant sa protection et son maintien. La fermeture du support est réalisée par un **système de coulissement**, permettant d'insérer ou de retirer facilement la caméra sans outil, tout en garantissant une bonne tenue lors du fonctionnement du robot.

La conception intègre également :

— des **orifices de fixation** permettant le montage du support sur l'avant-bras du robot Niryo,

— des **passages pour les câbles**, assurant l'alimentation et la communication de la caméra sans contrainte mécanique,

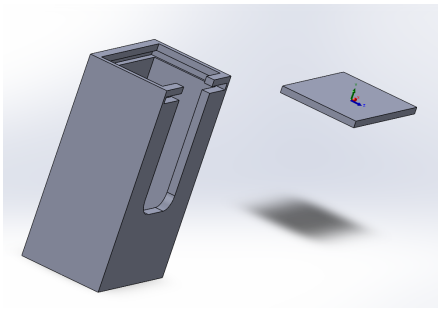
— une géométrie compacte afin de limiter l'impact sur la cinématique du bras robotique.

Cette conception simple et modulaire facilite l'intégration mécanique de la caméra tout en conservant une grande flexibilité pour d'éventuelles modifications futures.

### 4.2 Fabrication par impression 3D

La fabrication du support a été réalisée par impression 3D à l'aide d'une imprimante **Bambu Lab**. Ce type d'imprimante permet une réalisation rapide et précise des pièces, ce qui est particulièrement adapté au prototypage dans un contexte de projet académique.

L'impression 3D offre une grande flexibilité pour ajuster la conception du support en fonction des contraintes mécaniques et des essais réalisés sur le robot.



Modèle CAO (STL)



Support imprimé



Support avec caméra

FIGURE 12 – Étapes de conception et d'intégration du support de la caméra ESP32-CAM

### 4.3 Intégration sur le robot

Une fois imprimé, le support a permis l'intégration directe de la caméra sur le robot Niryo. Cette configuration garantit une position stable de la caméra et facilite l'acquisition d'images cohérentes pour les traitements de vision par ordinateur.

Cette étape constitue un lien essentiel entre la conception mécanique et les modules de perception visuelle décrits précédemment.

## 5 Bilan final du projet

Cette section présente un bilan global du travail réalisé, en mettant l'accent sur les avancées apportées depuis le premier état du projet et sur le niveau d'intégration atteint en fin de développement.

### 5.1 Travaux réalisés sur la phase finale

La phase finale du projet a permis d'aller au-delà d'un simple démonstrateur de détection isolé. Plusieurs briques complémentaires ont été ajoutées ou consolidées.

Tout d'abord, un **pré-traitement optique** a été intégré à la chaîne de vision par le biais d'une calibration de la caméra sous OpenCV. Cette étape a permis d'obtenir la matrice intrinsèque et les coefficients de distorsion, puis d'appliquer une correction géométrique aux images avant leur exploitation.

Ensuite, le **jeu de données a été enrichi** avec de nouvelles images acquises au cours du projet. Ces images ont été annotées manuellement, puis réinjectées dans le processus d'apprentissage. Le modèle de détection a ainsi été **réentraîné**, ce qui a permis d'améliorer sa robustesse et sa cohérence sur les scènes les plus proches des conditions réelles.

Enfin, la **chaîne complète perception-action** a été structurée et mise en œuvre : acquisition vidéo, détection de la cible, transformation géométrique par homographie, calcul de la position cible, puis transmission de la commande vers le robot.

## 5.2 Architecture finale obtenue

L'architecture finale du système repose sur une répartition claire des rôles :

— l'**ESP32-CAM** embarquée assure l'acquisition et l'envoi du flux vidéo ;

— le **PC central** reçoit ce flux par Wi-Fi et exécute les traitements de vision et de calcul ;

— le **robot Niryo** est commandé depuis ce même PC par liaison Ethernet.

Cette organisation permet de dissocier la vision embarquée, légère et mobile, du calcul principal, plus exigeant en ressources. Elle constitue une base cohérente pour un système robotique semi-déporté combinant perception visuelle et commande temps quasi réel.

## 5.3 Résultats obtenus

À l'issue du projet, plusieurs résultats concrets ont été obtenus :

— acquisition et transmission fonctionnelles d'un flux vidéo depuis l'ESP32-CAM ;

— calibration optique de la caméra et correction de la distorsion ;

— entraînement puis réentraînement d'un modèle YOLOv8 dédié à la détection des mauvaises herbes ;

— détection visuelle fonctionnelle sur des images proches de la scène réelle ;

— mise en place d'un pipeline complet reliant la détection à la commande du robot ;

— intégration mécanique de la caméra sur le bras via un support imprimé en 3D.

Le projet a donc permis de valider la faisabilité d'une approche combinant intelligence artificielle, vision par ordinateur, traitement géométrique et robotique sur un système unique.

## 5.4 Limites observées

Malgré ces résultats, certaines limites subsistent. Le débit du flux vidéo fourni par l'ESP32-CAM reste modeste et peut introduire de la latence dans la chaîne globale. La robustesse de la détection demeure dépendante des conditions d'éclairage et du contenu exact du jeu de données. De plus, la localisation repose sur une hypothèse de planéité qui simplifie volontairement le problème mais limite la généralisation à des scènes plus complexes.

Enfin, la stratégie de saisie reste fondée sur une commande relativement simple en altitude, sans retour visuel temps réel pendant le mouvement final de préhension.

## 5.5 Conclusion générale

Ce projet a permis de développer une chaîne fonctionnelle allant de l'acquisition d'une image jusqu'à la génération d'une commande exploitable par un bras robotique. Les avancées réalisées sur la seconde partie du projet, notamment le pré-traitement par calibration, l'enrichissement du dataset, le réentraînement du modèle et l'intégration de la pipeline complète, ont transformé un prototype partiel en un système global cohérent.

Au-delà des résultats techniques, ce travail a également permis de mettre en pratique des compétences complémentaires en intelligence artificielle, vision par ordinateur, géométrie projective, intégration logicielle et conception mécanique. Il constitue ainsi une base solide pour une poursuite éventuelle vers un système de désherbage encore plus autonome et plus robuste.